rootaccess2011's CrackMe with password

Introduction

This writeup details the analysis and solution of a Windows x64 crackme challenge created by rootaccess2011. The binary is heavily protected with multiple commercial packers and protectors, making static analysis challenging but not impossible.

Initial Reconnaissance

File Information

Using Ghidra, we can gather basic information about the binary:

- **Architecture**: x64 Windows PE executable
- **Entry Point**: `0x140001c10`
- **Protection**: Multiple layers (Enigma Protector, Themida, VMProtect, WinLicense)

Memory Segments

The binary contains several interesting segments:

```
.text:    140001000 - 1400025ff  (Standard code section)
.enigma1: 140003000 - 1400031ff  (Enigma Protector)
.enigma2: 140004000 - 1400041ff  (Enigma Protector)
.themida: 140005000 - 1400051ff  (Themida packer)
.vmp0:    140006000 - 1400061ff  (VMProtect)
.vmp1:    140007000 - 1400071ff  (VMProtect)
.rdata:   140008000 - 1400091ff  (Read-only data)
.data:    14000a000 - 14000a1ff  (Initialized data)
.winlice: 14020c000 - 14024bfff  (WinLicense)
.junk:    14024c000 - 14038bfff  (Junk/padding - over 1MB)
.vlizer:  14038e000 - 140688fff  (Very large section - 7MB+)
```

The presence of multiple protection layers and large junk sections indicate this is a heavily obfuscated binary designed to thwart reverse engineering efforts.

Imported Functions

Despite the heavy protection, the binary imports standard C runtime functions:

I/O Functions:

- `fgets` - Reading input
- `puts` - Displaying output
- `__stdio_common_vfprintf` - Formatted output
- `__acrt_iob_func` - File stream access
- `fflush` - Flushing output buffer

String Functions:

- `strcmp` - String comparison
- `strlen` - String length
- `strncpy_s` - Safe string copy

Console Functions:

- `_getch` - Reading single character without echo

This tells us the program likely reads user input, performs validation, and displays results.

String Analysis

Using Ghidra's string listing feature, we quickly identify the key strings:

```
0x140008240: "Enter Password: "
0x140008258: "\nNo input. Exiting.\n"
0x140008270: "Access granted."
0x140008280: "Access denied."
0x140008290: "\nPress Enter to exit..."
```

These strings confirm this is a password validation challenge with success/failure messages.

Control Flow Analysis

Entry Point

The entry point at `0x140001c10` is minimal:

```
void entry(void)
{
  FUN_14038e058();
  FUN_1405af34f();
  return;
}
```

These functions are likely unpacking/initialization routines within the protected sections.

Main Function

By tracing cross-references to the "Enter Password: " string, we find the main logic at `0x140001780`:

```
void FUN_140001780(int param_1, longlong param_2)
{
  // ... variable declarations ...

  // Anti-analysis initialization
  local_128 = 0xdeadbeef;
  iVar6 = 0;
  do {
    uVar4 = iVar6 * 0x9e;
    iVar6 = iVar6 + 1;
    local_128 = uVar4 ^ (local_128 ^ (local_128 << 3 | local_128 >> 5)) +
  } while (iVar6 < 3500000);  // Time-consuming loop

  // XOR obfuscation with multiple memory regions
  local_124 = DAT_14020c000 ^ (byte)local_128 ^ DAT_14000c000 ^ DAT_1401(
  DAT_14024c000 = DAT_14024c000 ^ local_124;

  // Call various protected functions
  FUN_140006000();
  FUN_140007000();
  FUN_140003000();
  FUN_140004000();
  FUN_140005000();

  // Handle command-line or interactive input
  if (param_1 == 2) {
    strncpy_s(acStack_119 + 1, 0x100, *(char **)(param_2 + 8), 0xffffffff
  }
  else {
```

```
    FUN_140001010("Enter Password: ");
    pFVar2 = (FILE *)__acrt_iob_func(0);
    pcVar5 = fgets(acStack_119 + 1, 0x100, pFVar2);

    if (pcVar5 == (char *)0x0) {
      FUN_140001010("\nNo input. Exiting.\n");
      goto LAB_1400018c0;
    }

    // Strip trailing newlines
    sVar3 = strlen(acStack_119 + 1);
    while ((sVar3 != 0 && ((acStack_119[sVar3] == '\n' || (acStack_119[sV
      acStack_119[sVar3] = '\0';
      sVar3 = sVar3 - 1;
    }
  }

  // Validate the password
  iVar6 = FUN_1400013f0(acStack_119 + 1);

  // Display result
  pcVar5 = "Access granted.";
  if (iVar6 == 0) {
    pcVar5 = "Access denied.";
  }
  puts(pcVar5);

  // Wait for user to press Enter
  FUN_140001010("\nPress Enter to exit...");
  pFVar2 = (FILE *)__acrt_iob_func(1);
  fflush(pFVar2);
  do {
    iVar6 = _getch();
    if (iVar6 == 0xd) break;
  } while (iVar6 != 10);

  // ... cleanup ...
}
```

Key Observations:

1. The function contains a time-wasting loop to slow down analysis
2. Multiple XOR operations across different memory regions for obfuscation
3. Calls to protected functions in various sections (VMProtect, Themida, etc.)

4. Password validation happens in `FUN_1400013f0`

Password Validation Function

The critical function `FUN_1400013f0` at `0x1400013f0` contains the password validation logic:

```
void FUN_1400013f0(char *param_1)
{
  // ... variable declarations ...

  // The actual password stored in memory!
  local_1048[0] = 0x72;   // 'r'
  local_1048[1] = 0x6f;   // 'o'
  local_1048[2] = 0x6f;   // 'o'
  local_1048[3] = 0x74;   // 't'
  local_1048[4] = 0x61;   // 'a'
  local_1048[5] = 99;     // 'c' (decimal)
  local_1048[6] = 99;     // 'c' (decimal)
  local_1048[7] = 0x65;   // 'e'
  local_1048[8] = 0x73;   // 's'
  local_1048[9] = 0x73;   // 's'
  local_1048[10] = 0x31;  // '1'
  local_1048[11] = 0x33;  // '3'
  local_1048[12] = 0x33;  // '3'
  local_1048[13] = 0x37;  // '7'
  local_1048[14] = 0;     // null terminator

  // Length check
  sVar8 = strlen(param_1);
  if ((int)sVar8 != 0xe) goto LAB_140001752;  // Must be 14 characters

  // Expected hash values
  local_1038 = 0xbdeceac9;
  local_1034 = 0xe3f0eeaa;
  local_1030 = 0x255df9d7;
  local_102c = 0x2527;

  // Transform input through XOR function
  FUN_140001070(param_1, local_1028);

  // FNV-1a hash with custom modifications
  uVar12 = 0xcbf29ce484222325;  // FNV-1a offset basis
  iVar7 = 0;
```

```
  do {
    iVar13 = iVar7 + 1;
    uVar12 = (uVar12 ^ local_1048[iVar7]) * 0x100000001b3;  // FNV-1a pri
    bVar9 = (char)iVar7 + (char)(iVar7 / 0xd) * -0xd + 3U & 0x3f;
    uVar12 = uVar12 << bVar9 | uVar12 >> 0x40 - bVar9;  // Rotate
    iVar7 = iVar13;
  } while (local_1048[iVar13] != 0);

  // ... complex VM-like bytecode generation and execution ...

  // Final comparison
  iVar7 = FUN_1400011c0(local_818, uVar12, &local_1038, local_1028);
  if (iVar7 != 0) {
    strcmp(param_1, (char *)local_1048);  // Direct comparison!
  }

  // ... cleanup ...
}
```

Critical Finding: The password is hardcoded in plaintext in the `local_1048` array!

Password Extraction

Converting the hex values to ASCII:

```
password_bytes = [
    0x72, 0x6f, 0x6f, 0x74,  # "root"
    0x61, 99, 99, 0x65,      # "acce"
    0x73, 0x73, 0x31, 0x33,  # "ss13"
    0x33, 0x37, 0            # "37\0"
]

password = ''.join(chr(b) for b in password_bytes if b != 0)
print(password)  # "rootaccess1337"
```

Additional Analysis

XOR Transformation Function

The function `FUN_140001070` at `0x140001070` performs a simple XOR transformation on the input:

```
void FUN_140001070(byte *param_1, byte *param_2)
{
  sVar2 = strlen((char *)param_1);
  iVar1 = (int)sVar2;

  if (iVar1 >= 1)  param_2[0] = param_1[0] ^ 0xbb;
  if (iVar1 >= 2)  param_2[1] = param_1[1] ^ 0x85;
  if (iVar1 >= 3)  param_2[2] = param_1[2] ^ 0x83;
  if (iVar1 >= 4)  param_2[3] = param_1[3] ^ 0xc9;
  if (iVar1 >= 5)  param_2[4] = param_1[4] ^ 0xcb;
  if (iVar1 >= 6)  param_2[5] = param_1[5] ^ 0x8d;
  if (iVar1 >= 7)  param_2[6] = param_1[6] ^ 0x93;
  if (iVar1 >= 8)  param_2[7] = param_1[7] ^ 0x86;
  if (iVar1 >= 9)  param_2[8] = param_1[8] ^ 0xa4;
  if (iVar1 >= 10) param_2[9] = param_1[9] ^ 0x8a;
  if (iVar1 >= 11) param_2[10] = param_1[10] ^ 0x6c;
  if (iVar1 >= 12) param_2[11] = param_1[11] ^ 0x16;
  if (iVar1 >= 13) param_2[12] = param_1[12] ^ 0x14;
  if (iVar1 >= 14) param_2[13] = param_1[13] ^ 0x12;

  // Fill remaining with 0xff
  // ...
}
```

Each character is XORed with a specific byte value. This transformed data is later used for validation.

VM-Like Checker

The function `FUN_1400011c0` at `0x1400011c0` implements a complex virtual machine-like checker:

```
bool FUN_1400011c0(longlong param_1, ulonglong param_2,
                   longlong param_3, longlong param_4, byte param_5)
{
  // Implements a bytecode interpreter
  // Opcodes include:
  // 0x20 - Load operation
  // 0x13 - Multiply operation
  // 0x14 - Rotate operation
  // 0x15 - Add to accumulator
  // 0xc0 - Toggle mode
  // 0xf0 - Check if accumulator is zero (success condition)
```

```
    // ... complex bytecode execution ...

    if (bVar2 == 0xf0) {
      return iVar9 == 0;  // Success if accumulator is zero
    }

    // ...
}
```

This VM adds an extra layer of obfuscation, but ultimately the direct `strcmp` call makes it unnecessary to fully understand.

Anti-Analysis Techniques Employed

1. **Multiple Commercial Packers**: Enigma, Themida, VMProtect, WinLicense
2. **Large Junk Sections**: Over 8MB of padding/junk data
3. **Time-Wasting Loops**: 3.5 million iteration loop to slow analysis
4. **XOR Obfuscation**: Multiple XOR operations across memory regions
5. **Custom VM**: Bytecode interpreter for validation logic
6. **FNV Hash with Rotation**: Modified FNV-1a hash algorithm
7. **Stack Cookie Protection**: Stack canary checks
8. **Indirect Calls**: Function pointers and dispatch tables

Despite all these protections, the password was stored in plaintext, making the challenge solvable through static analysis alone.

Solution

Password: `rootaccess1337`

Verification

Running the crackme and entering the password:

```
Enter Password: rootaccess1337
Access granted.

Press Enter to exit...
```

Conclusion

This crackme demonstrates an interesting paradox: heavy protection with multiple commercial packers and anti-analysis techniques, yet the password is stored in plaintext within the validation function. This suggests the challenge was designed to test reverse engineering skills against protected binaries rather than cryptographic strength.

The key lessons from this analysis:

1. **Multiple protections don't guarantee security** - If the core logic is flawed, layers of protection are meaningless
2. **Static analysis can succeed** - Even against heavily packed binaries, patient analysis can reveal secrets
3. **Ghidra is powerful** - Modern decompilers can handle obfuscated code reasonably well
4. **Look for the obvious first** - Sometimes the simplest approach (finding hardcoded strings) works best

---